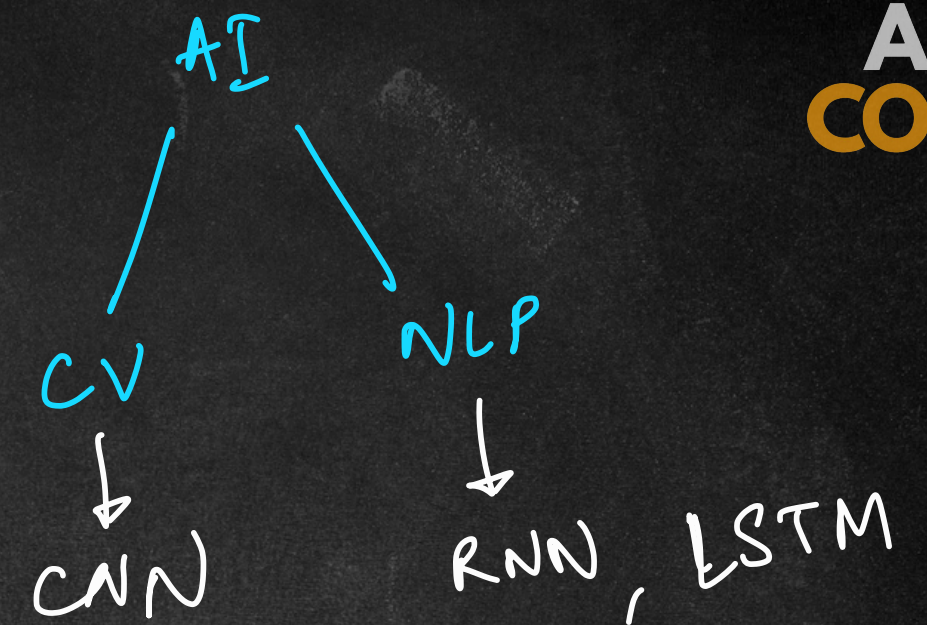
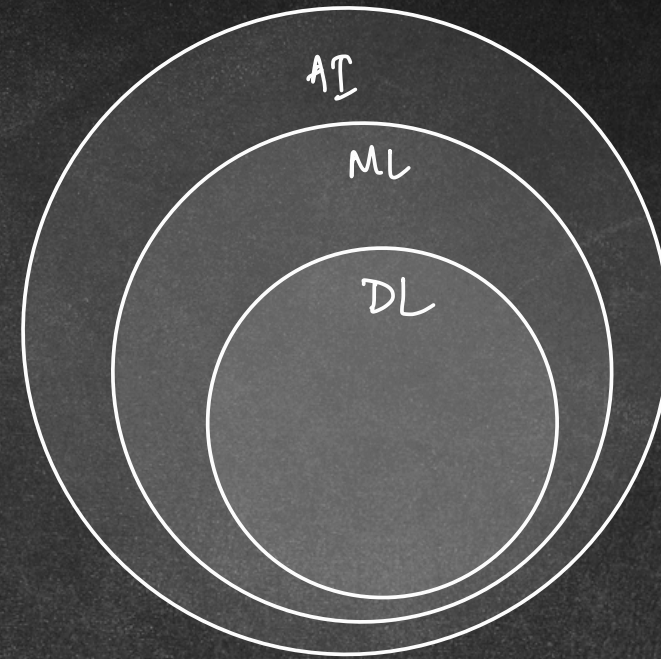


GenAI

Generative AI

↓
create new content

ANN
RNN → LSTM
CNN
Transformer



images/videos → dalle., midjourney
text → chatgpt, claud, gpt
audio/sound/music → sound draw
code → copilot
:
:
:

LLMs

Large Language Models

✓ Billions of parameters

class of models in NLP

✓ trained on huge datasets

✓ capabilities ↑↑

How are you? ✓

Q: why should I code in Python?

Ans: Python is a great language

GenAI



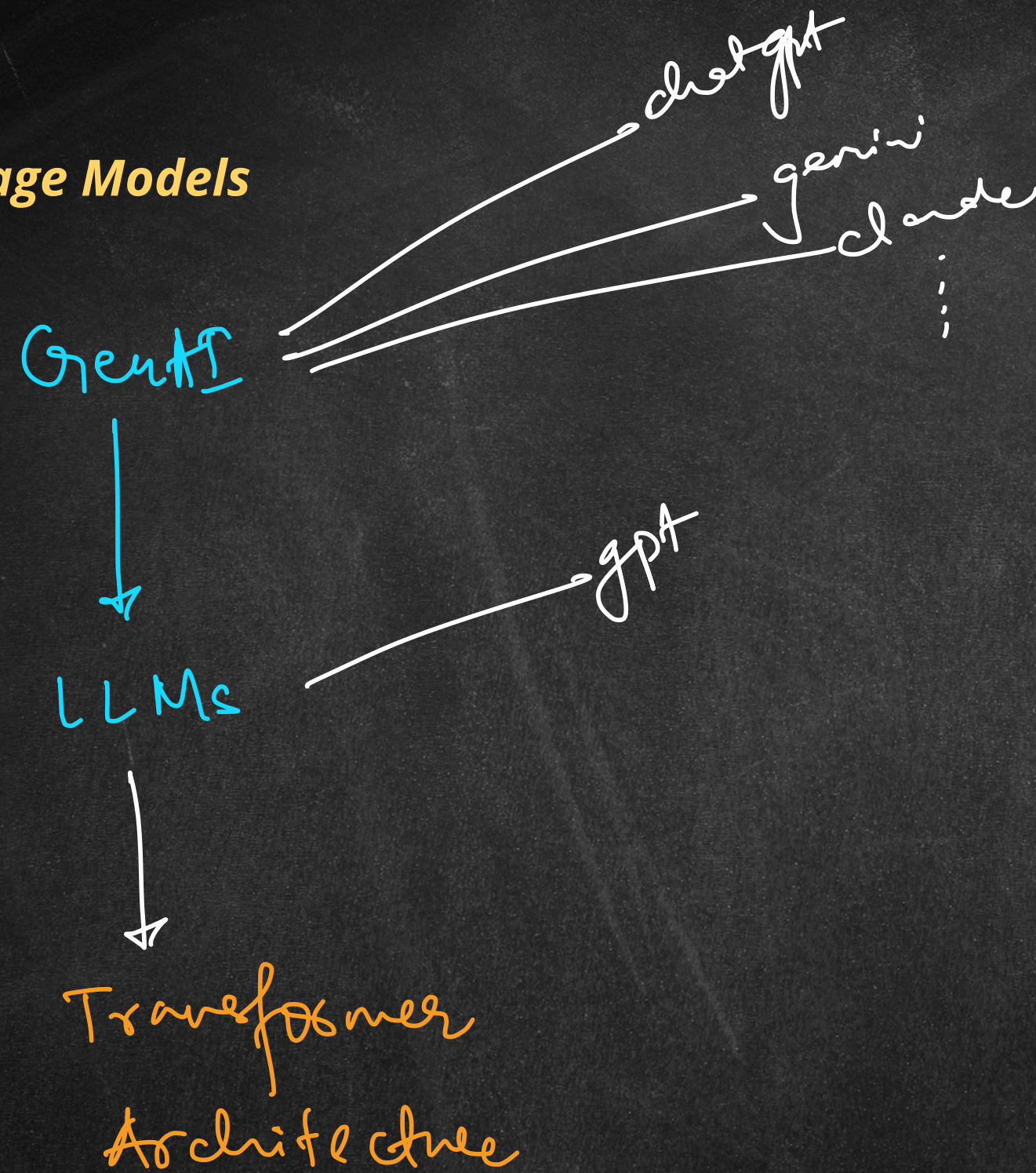
text / language

LLMs

APNA COLLEGE

LLMs

Large Language Models



Transformers

↓
NN arch.

2017 ⇒ Attention is all you need

⇓
Attention ⇒ parallelly

Sequence → text
"I like Python"

↓
RNN, LSTM, CNN

step by step (sequentially)

↓
sequence models

Transformers

Embedding $\rightarrow$ RNN

word embeddings $\rightarrow$ vectors

text: "I like Python"

nums

I, like, Python

[0.1, 0.7]

[0.32, 0.41]

[0.8, 0.2]

Is it a language?
py ✓

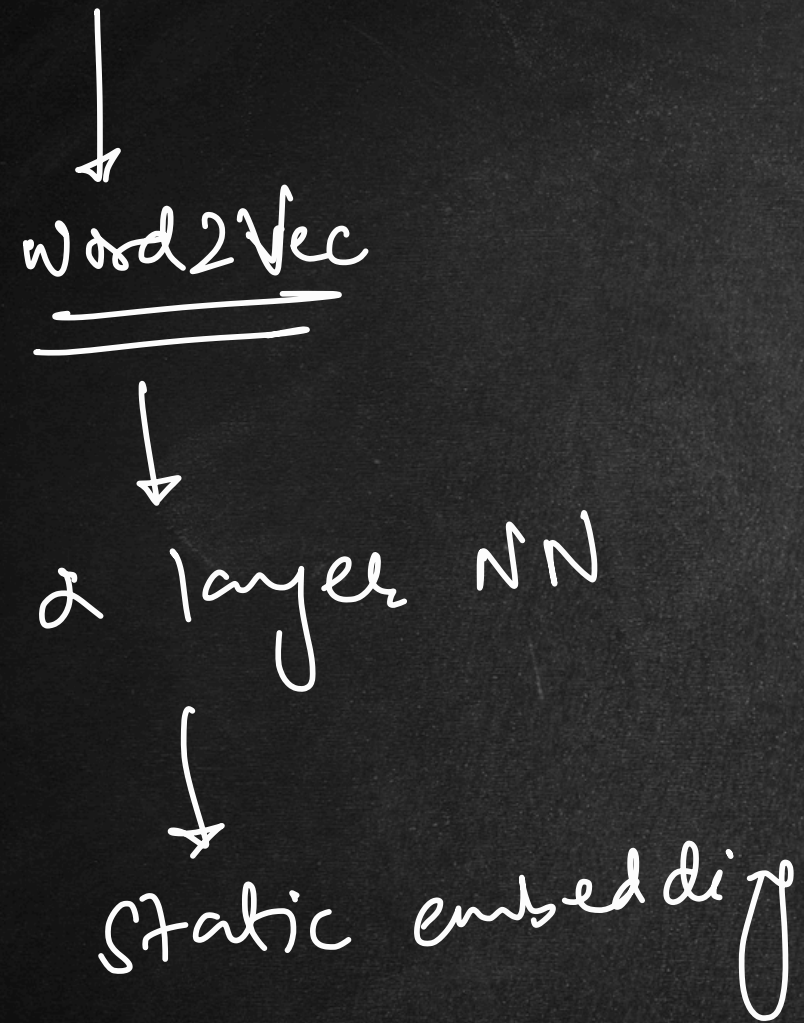
Is it an object?

features

gpt-3 $\rightarrow$ 12k $\rightarrow$ 12,288 dimensions in embeddings

Transformers

Static vs Contextual Embedding



I will read a book.

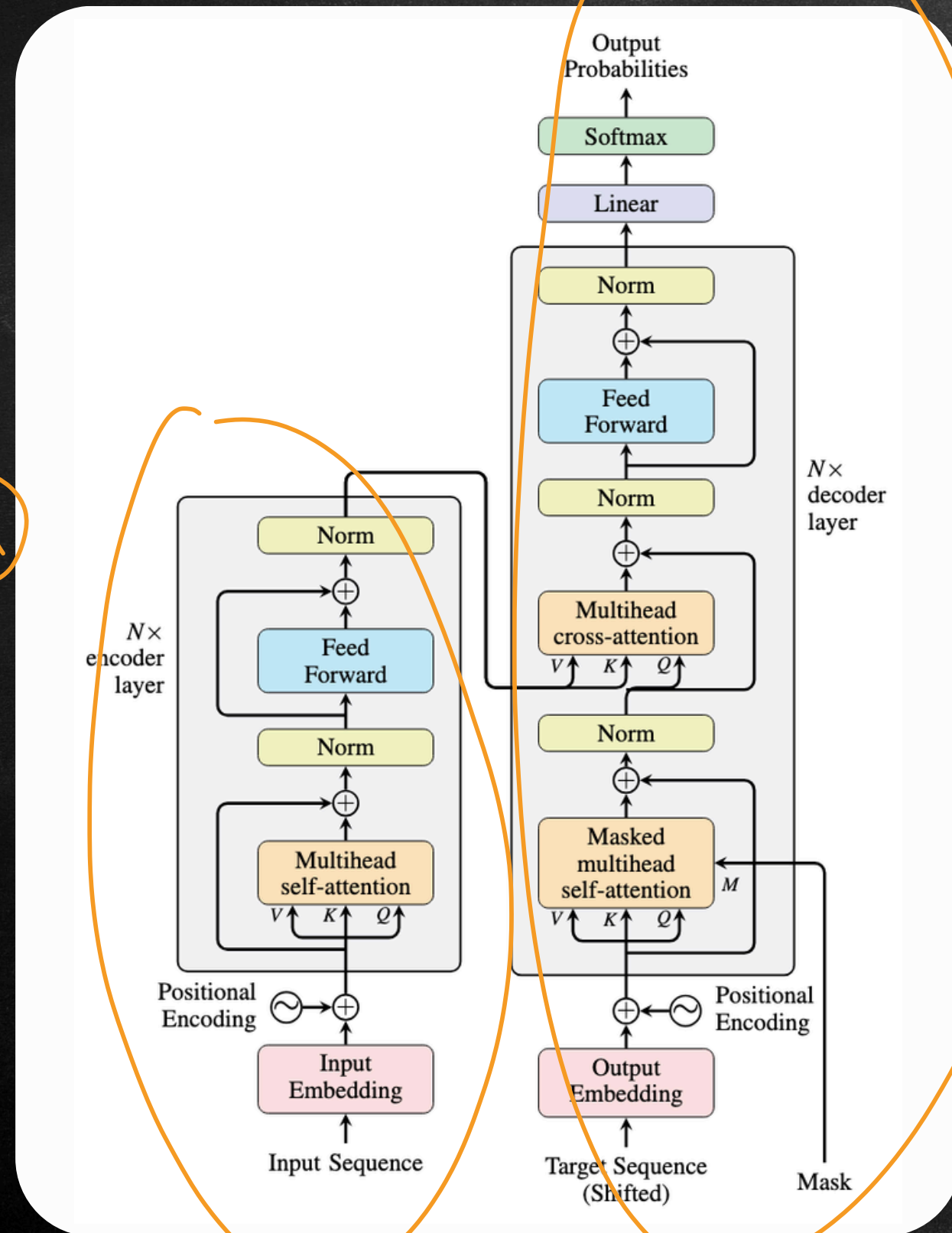
I will book movie tickets.

book $\rightarrow [0.32, 0.5, 0.01]$

python $\rightarrow$ snake
 $\searrow$ prog. language

Transformers

Architecture



② $\frac{9}{10}$ $\frac{10}{10}$ $\frac{10}{10}$ $\frac{10}{10}$

① encoder understand
② decoder generate

"I like Python"
inp sequence

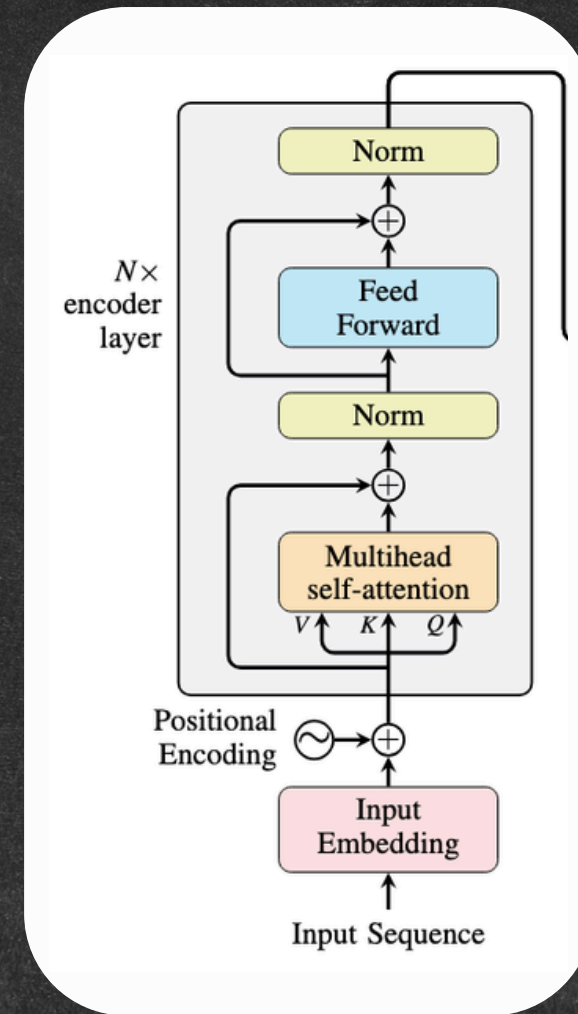
⇒ contextual
embeddings

I
liked
Python

- ① summarize
- ② sentiment analysis
- ③ text classification
- ④ speech translation

- ① Tokenization
- ② Contextual embeddings
- ③ pass to the decoder

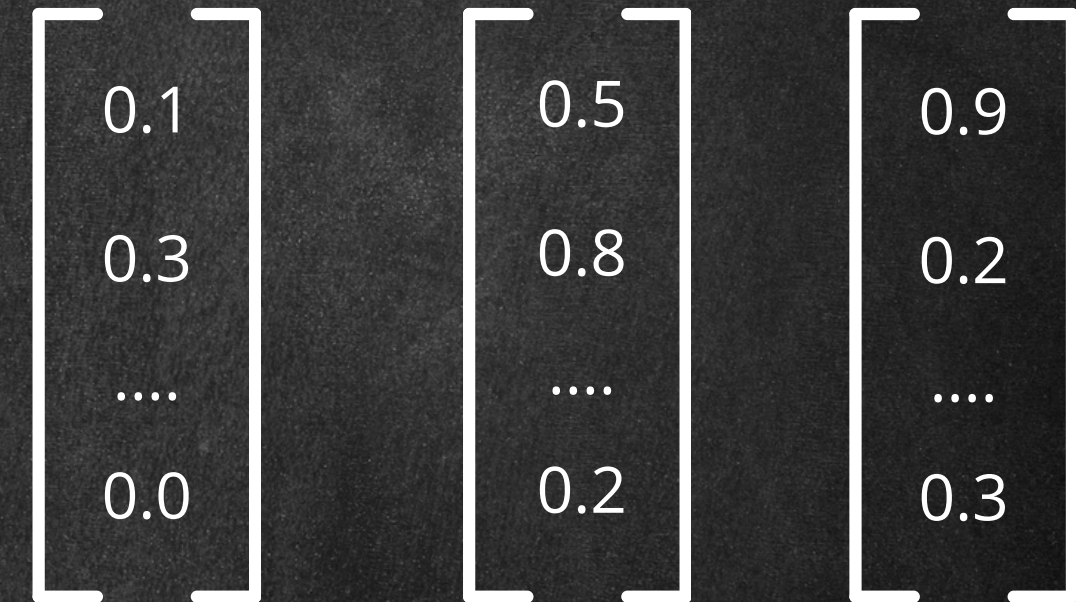
"I like Python"



Encoder

"I like Python"

[I, like, Python]



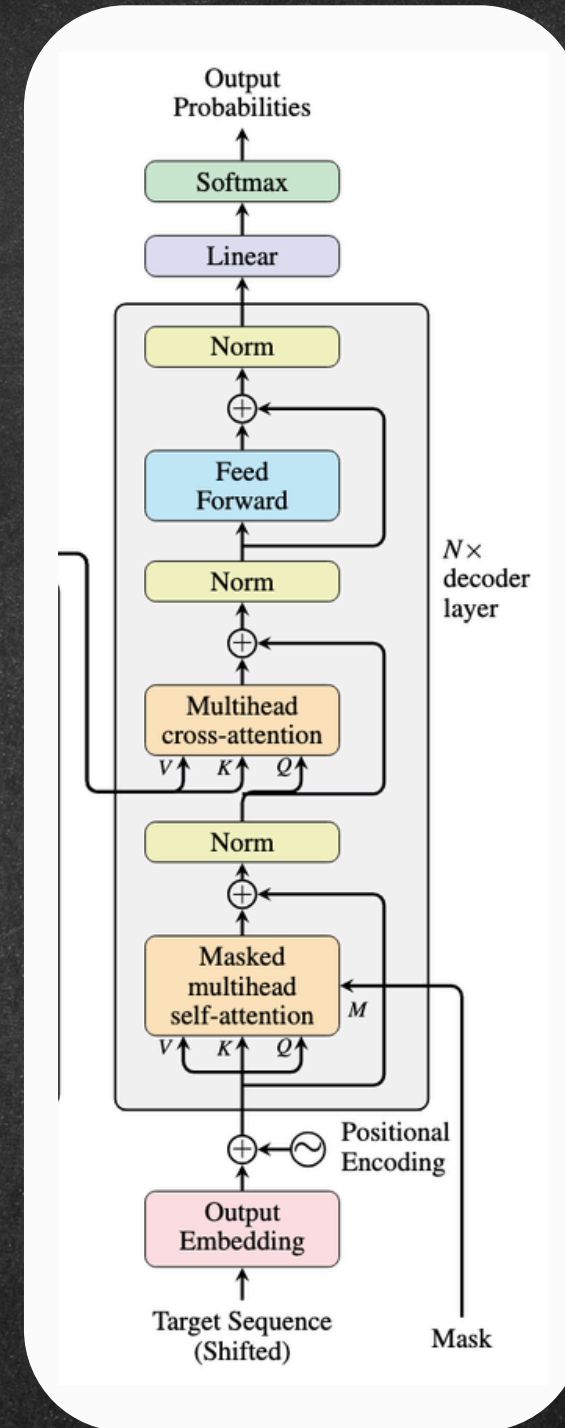
Static embedding $\rightarrow$ contextuale
+
vocab

"I like Python"

[I, like, Python]

0.1	0.5	0.9
0.3	0.8	0.2
....		
0.0	0.2	0.3

generate o/p one at a time



Decoder

<START>

मुझे

पाइथन

पसंद

है

<EOS> end of sequence

APNA COLLEGE

72%

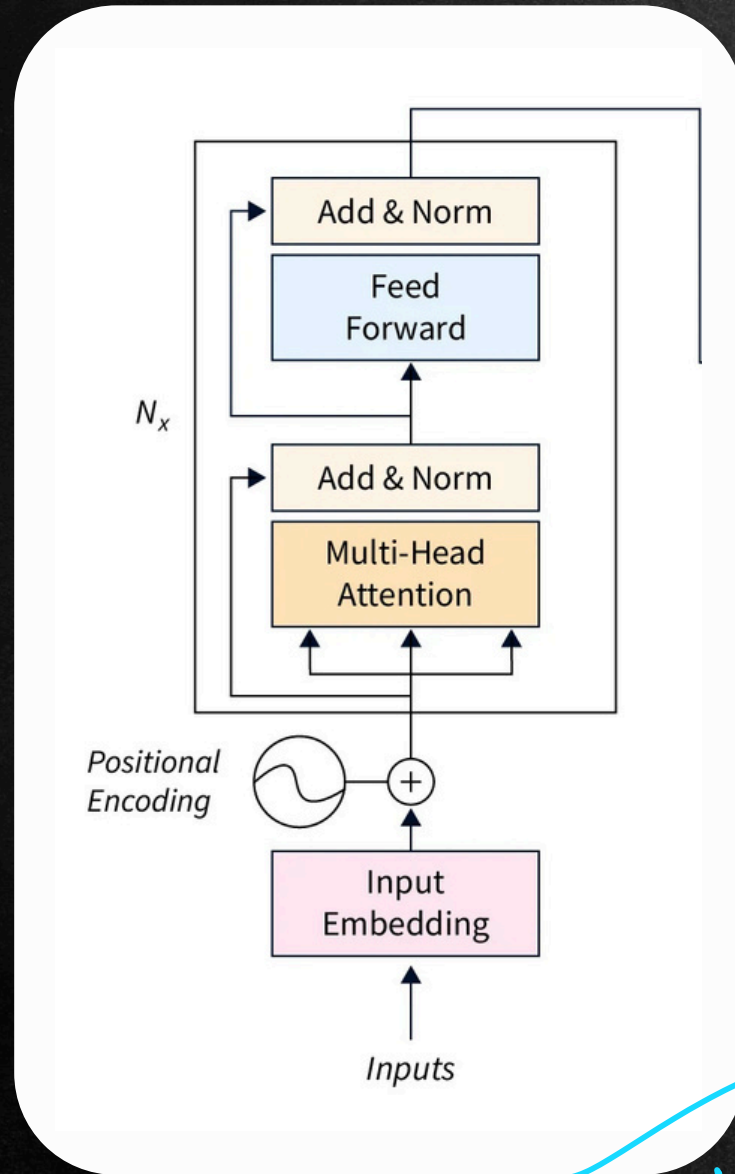
10%

0.2%

0.05%

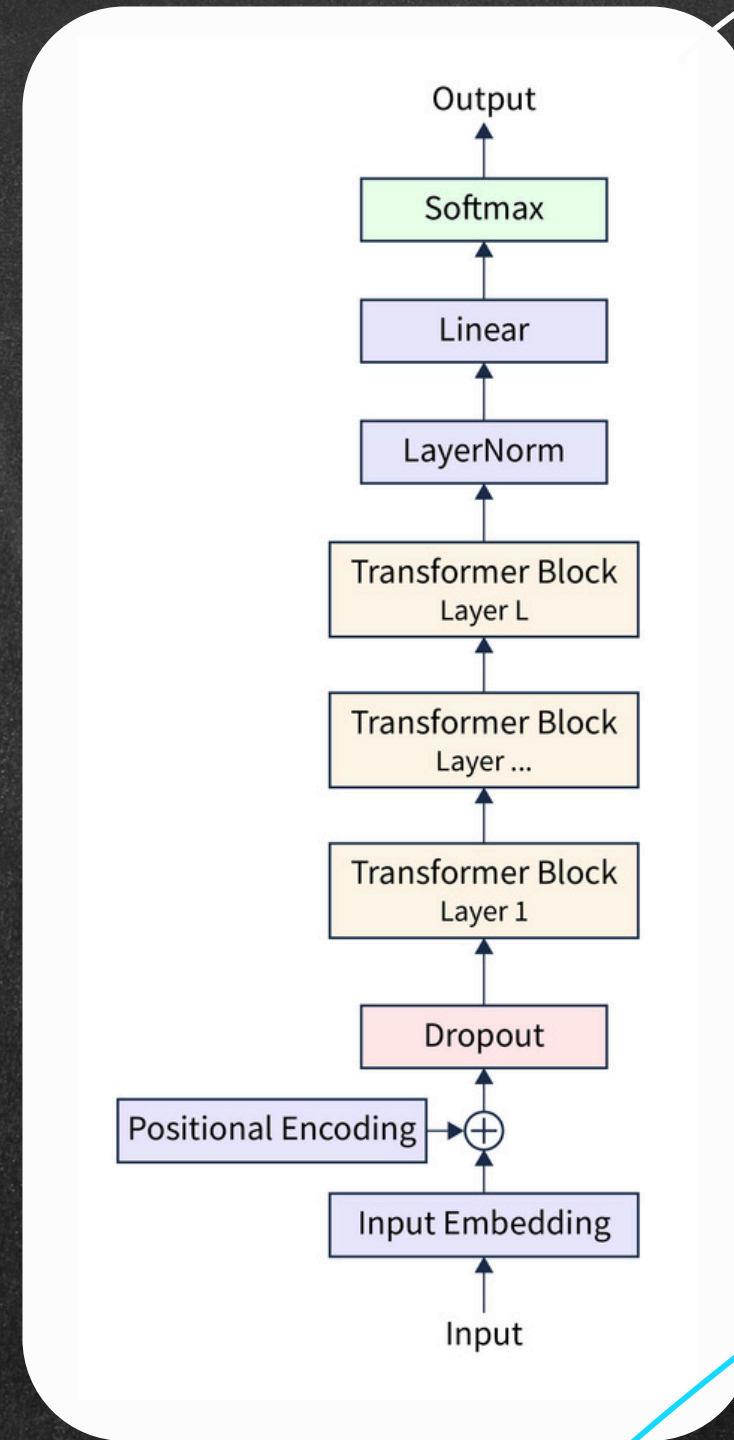
Different Transformer Architectures

- text classification
- sentiment analysis



BERT

encoder - only
transformer arch.



GPT / GPT-2

decoder - only
transformer arch.

① conversation

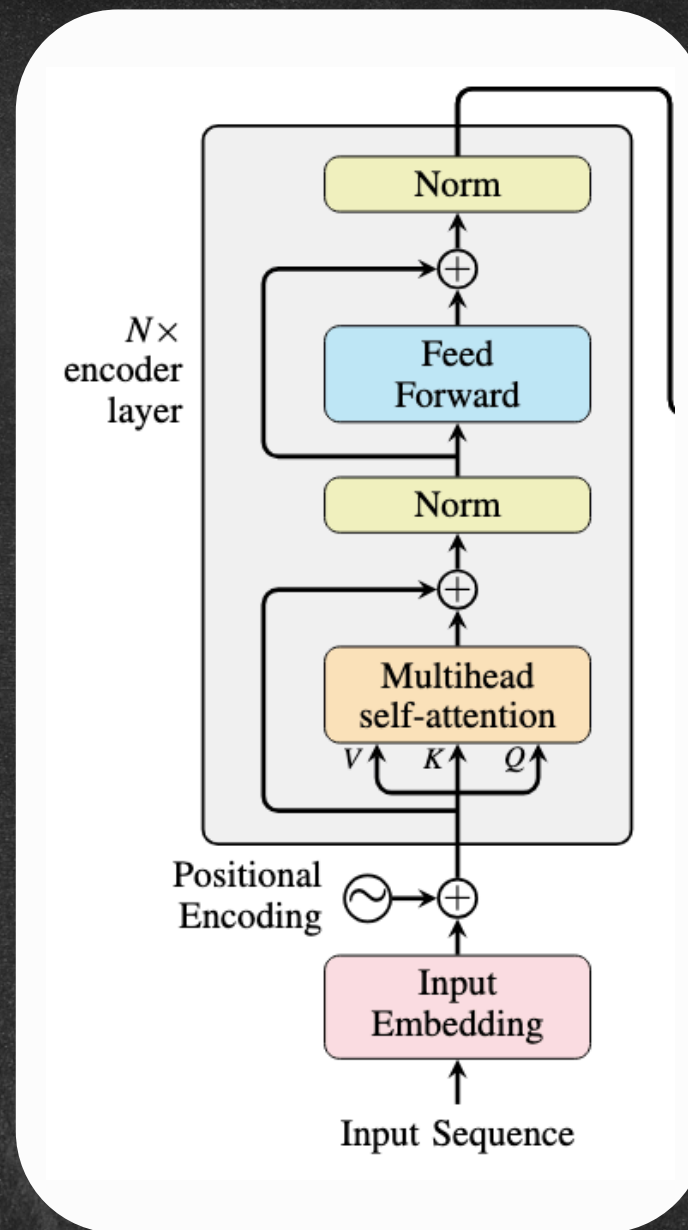
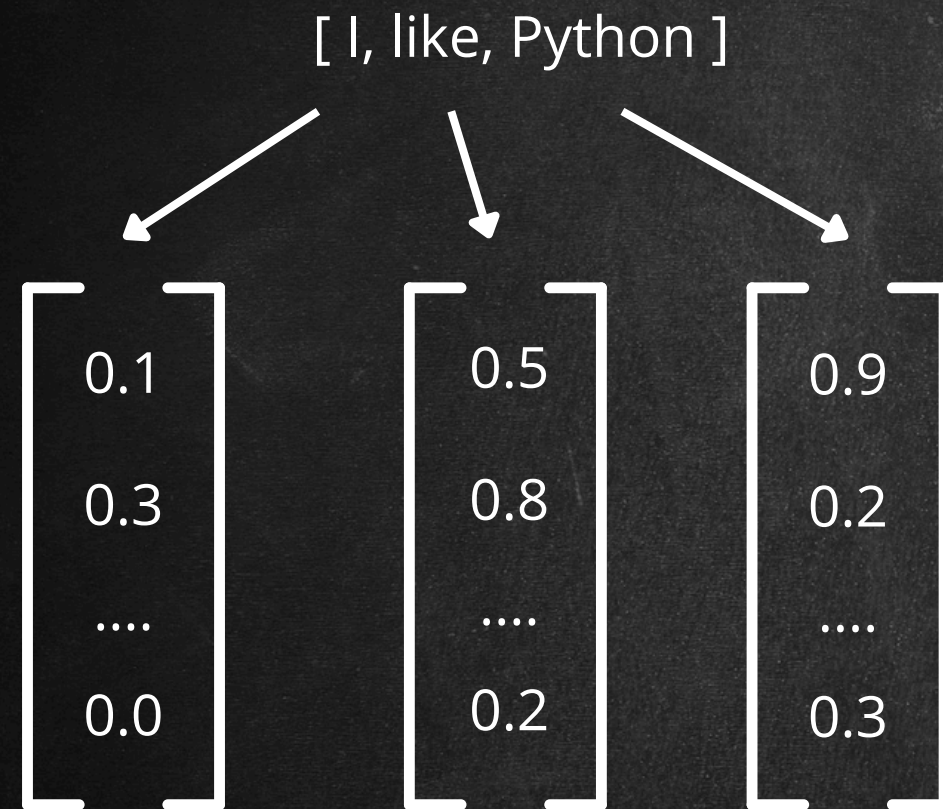
② easier to train

token by token

text generation

Encoder

Positional Encoding



I like Python
Python like I

RNN → sequentially
↑
I, like, Python

Trans for mtl
↓
Parallelization
— training faster
— performance ↑

Encoder

Positional Encoding

[I, like, Python]

0.1
0.3
....
0.0

0.5
0.8
....
0.2

0.9
0.2
....
0.3

+

+

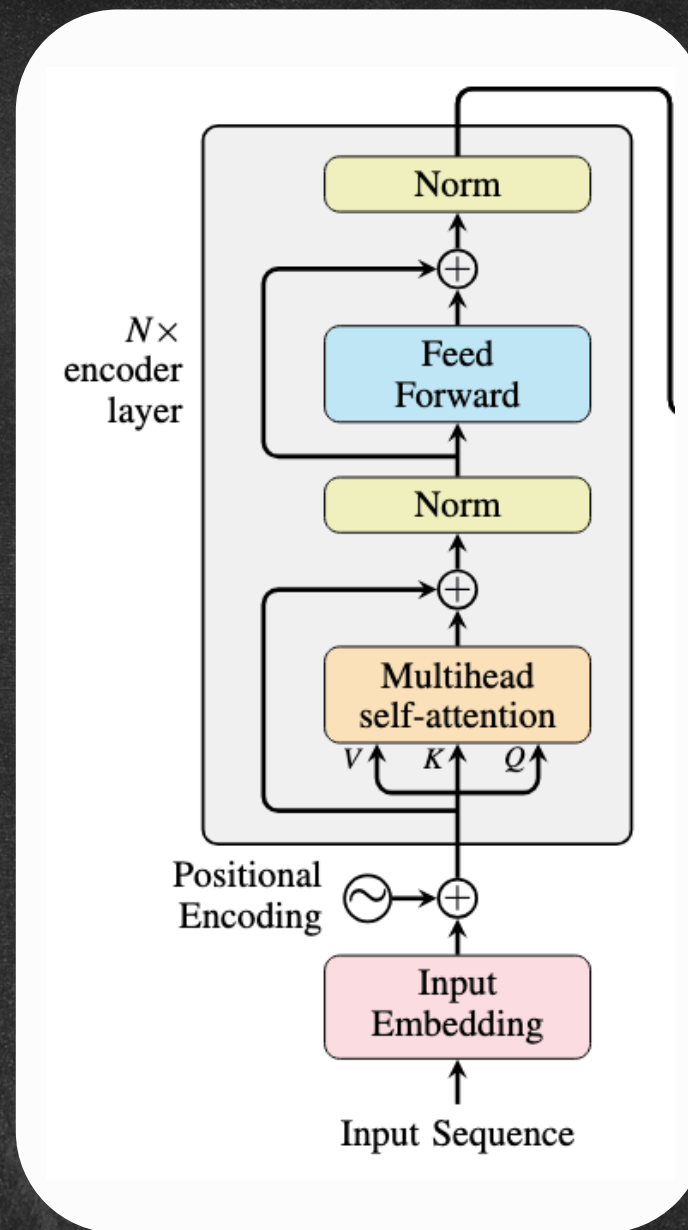
+

0.5
0.1
..
0.3

[]

[]

0.6
0.4
..
0.3



final $\frac{0}{p}$ = $\frac{1}{p}$ embeddy + position embeddings

$$PE(pos, 2i) = \sin\left(\frac{pos}{10,000^{2i/d_{model}}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10,000^{2i/d_{model}}}\right)$$

Attention

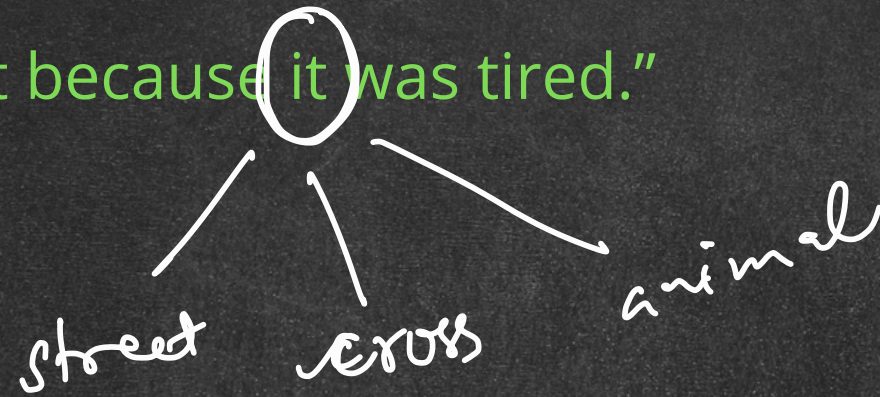
Attention is all you need.

focus

what/who should it pay attention to?

APNA
COLLEGE

"The animal didn't cross the street because it was tired."



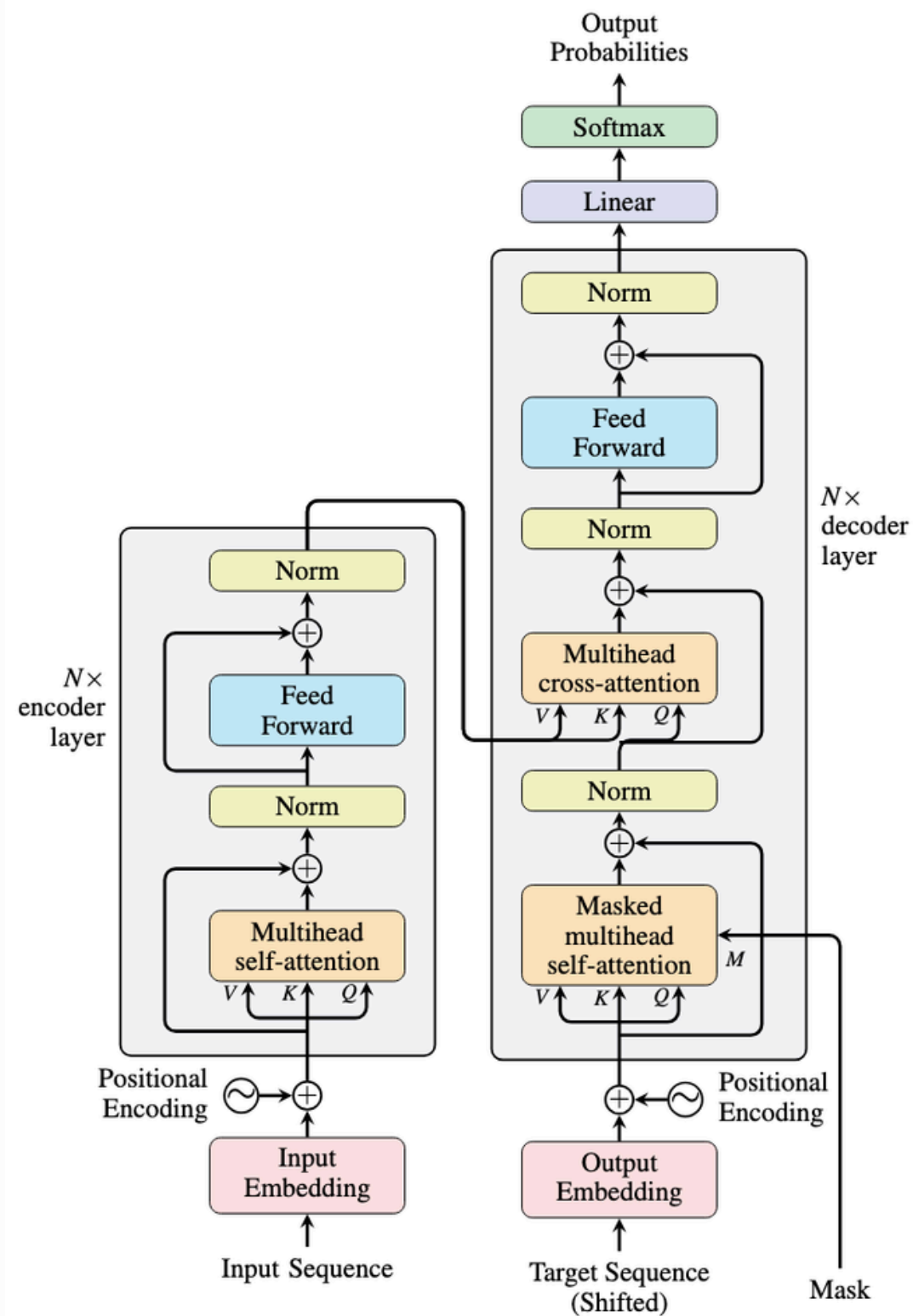
mapping

Q → K, V

Attention

Attention is all you need.

Attention



Attention

Attention is all you need.

Teacher $\rightarrow$ Transformer

Query (Q): what is the capital of India?

R: geography (Key) $\rightarrow$ V: capital of India

S: math (Key) $\rightarrow$ V: $2+2=4$

Gr: history (Key) $\rightarrow$ V: Independence/Constitution of India

strong match

= 0.8

low = 0.05

med. = 0.1

3 matrices

Q, K, V

attention score $\Rightarrow$ scaled dot product
 $\text{Attention}(Q, K, V)$

Attention

Attention is all you need.

input seq $\rightarrow$ n token
 $d \rightarrow$ dimension of model

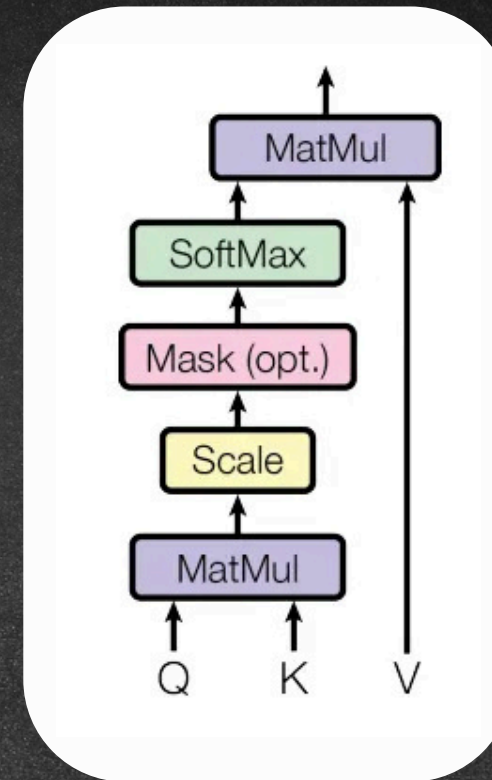
① $X = \begin{bmatrix} 0.3 \\ 0.4 \\ 1.1 \\ 0.1 \end{bmatrix} \Rightarrow n \times d$

$$Q = X \cdot W_Q \rightarrow d \times d_k$$

$$K = X \cdot W_K \rightarrow d \times d_k$$

$$V = X \cdot W_V \rightarrow d \times d_v$$

$\searrow$ weight matrices



Scaled Dot Product
Attention

$\Downarrow$
Attention(Q, K, V)

Attention

Attention is all you need.

Q, K, V

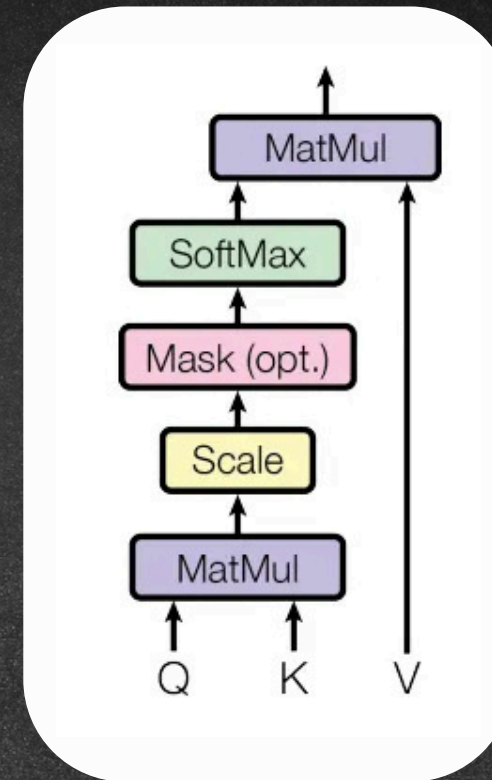
(dot product)

① similarity score $\Rightarrow Q \cdot K^T$

② scaling $\Rightarrow \frac{Q \cdot K^T}{\sqrt{d_k}}$

③ softmax $\Rightarrow \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
↳ probabilities

④ Attention $(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$



geography — 0.8
math $\rightarrow$ 0.05

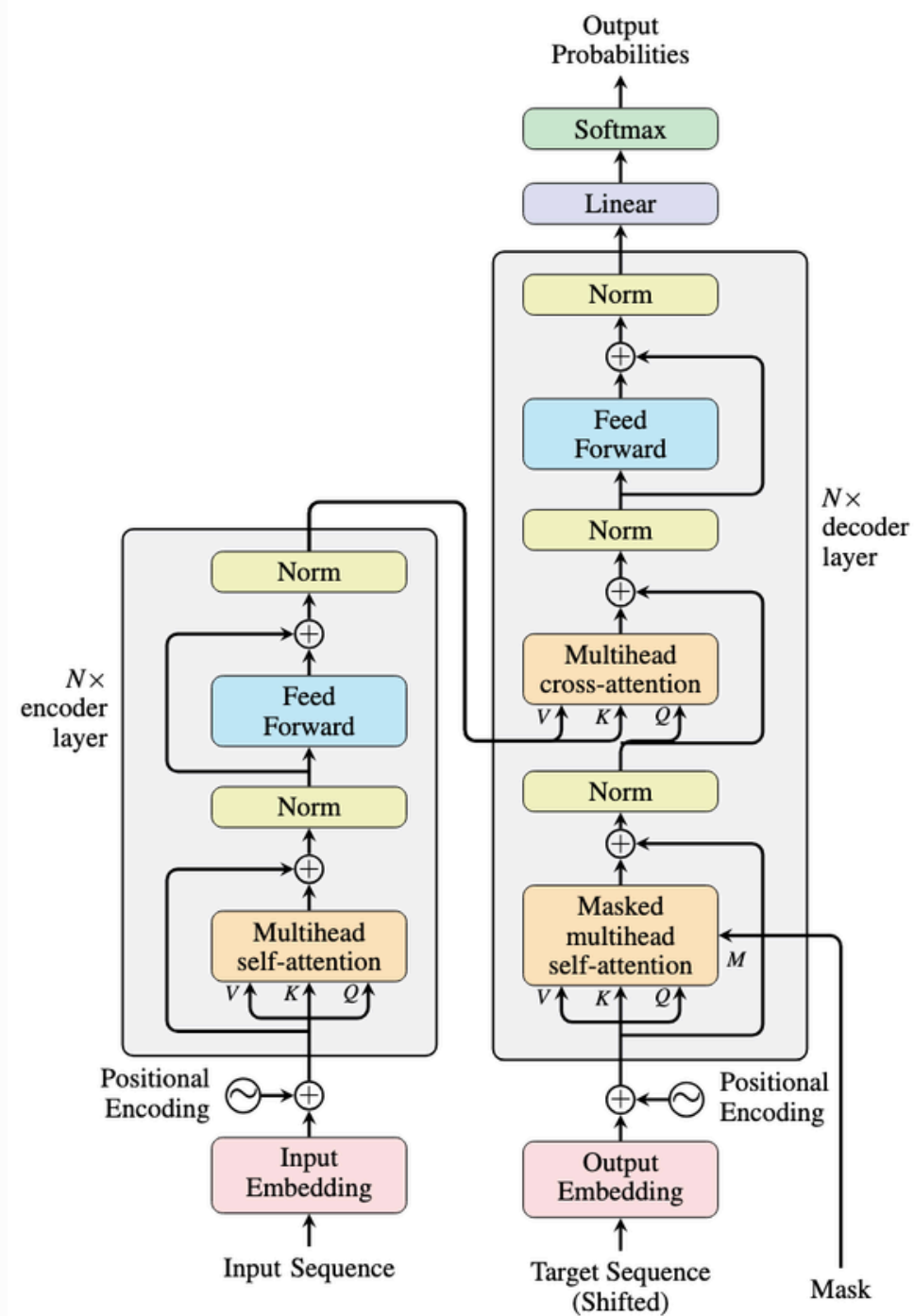
APNA
COLLEGE

Attention

Self-Attention

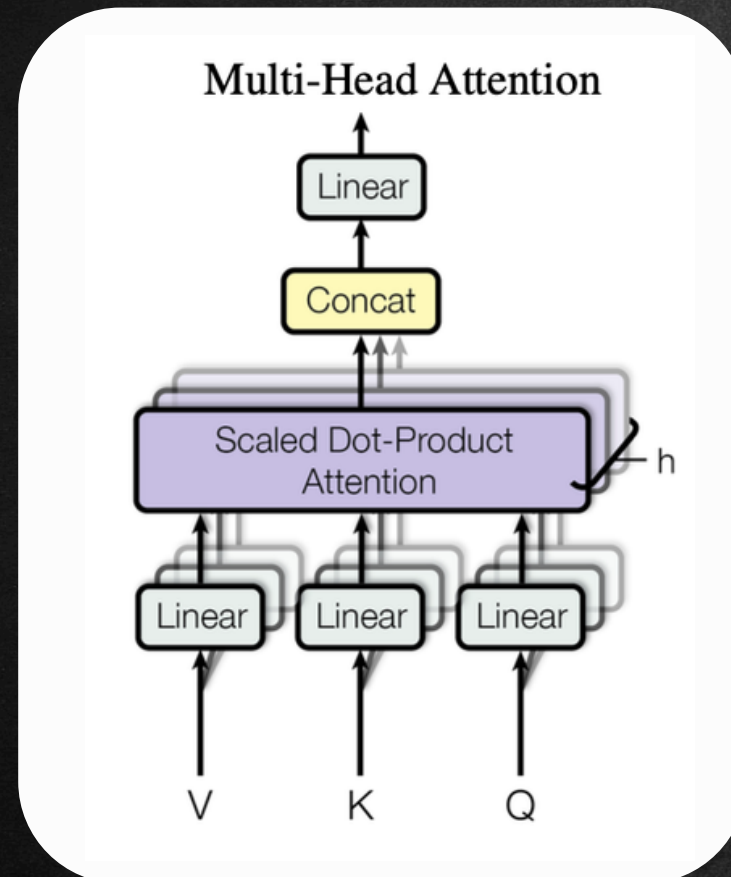
(SDP) scaled dot product $\rightarrow$ self - Attention
 $\downarrow$
contextual embeddings

single head self-attention
i/p embedd $\rightarrow$ positional encoding $\rightarrow$ SDP $\rightarrow$ contextual embedd



Attention

Multi-Head Attention



multiple heads
⇓

"I like python." → if p embed → positional modif
→ AN1 → CE1
→ AN2 → CE2
→ AN3 → CE3
⋮

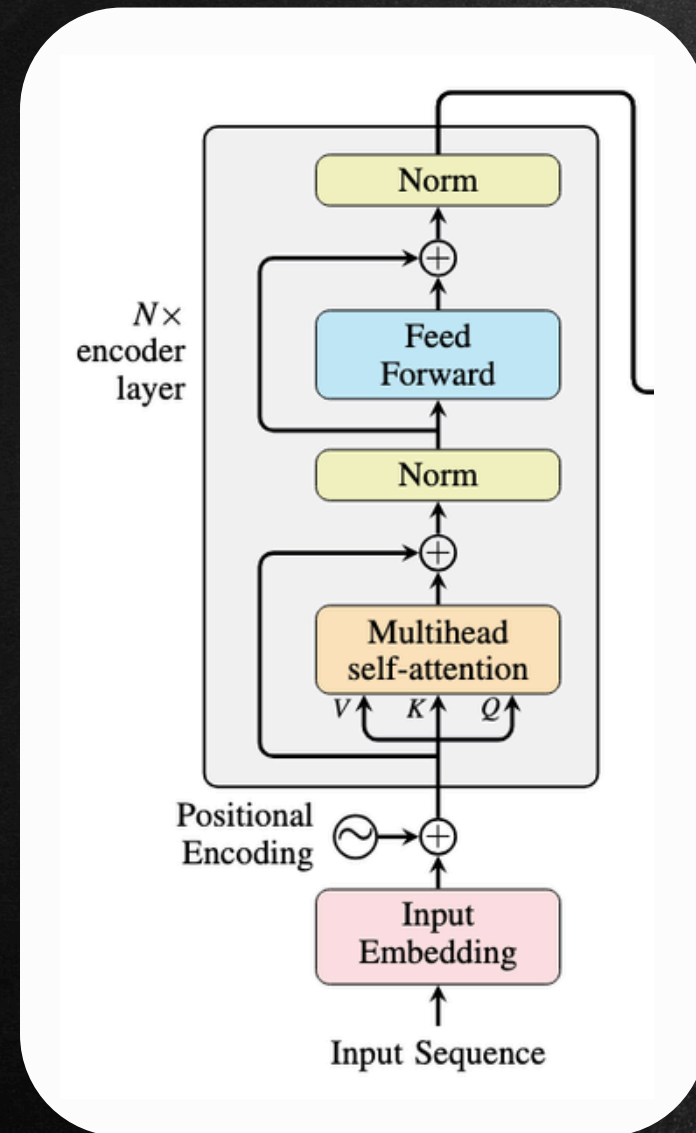
grammar

meaning

- ① self-attention
- ② cross-attention
- ③ masked attention

Encoder

Residual Connection



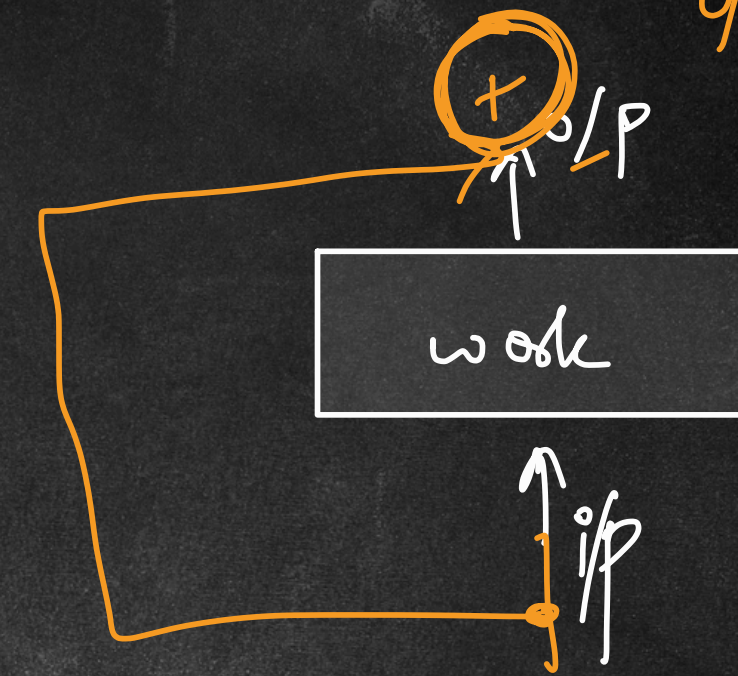
raw
inp $\xrightarrow{\text{directly connect}}$ output

Add + Norm

① vanishing gradient

② distorted info. in NN connection

↓
preserve original



Normalization

faster convergence
↓
stabilize training

Encoder

FNN

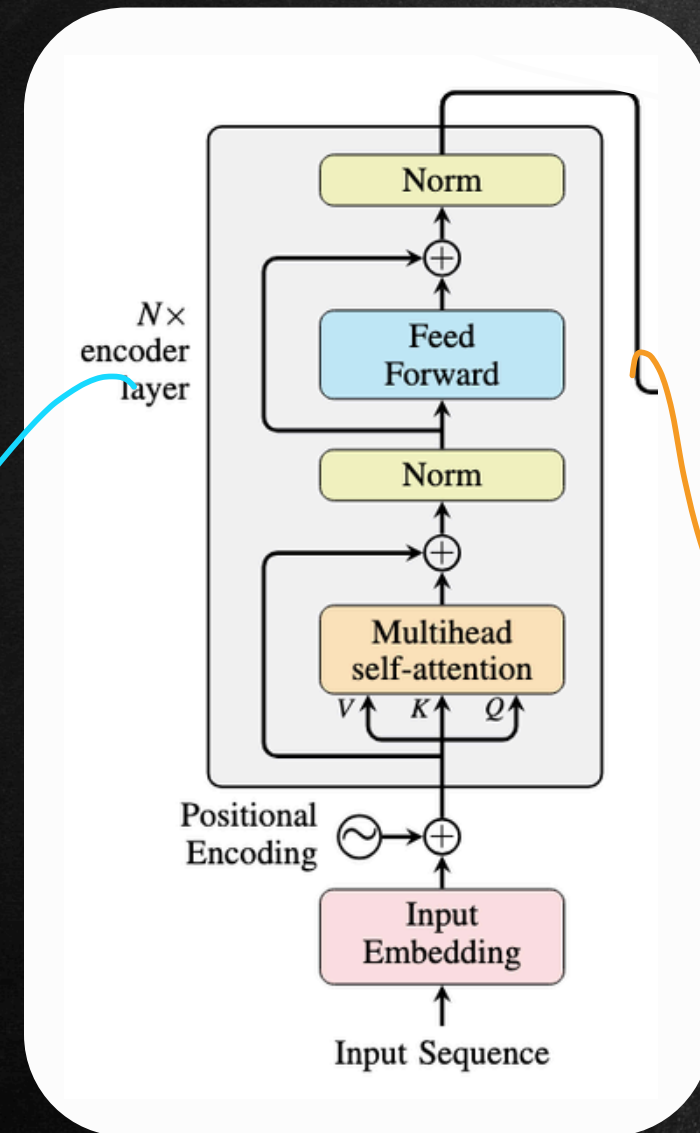
adds non-linearity



" complexity

more
enriched
understanding

contextual
embedd.



① gather info.

② think deeply info.

imp features ↑↑

irrelevant
info ↓↓

gp2 → N = 48
gp3 → N = 96

Decoder

Cross-Attention

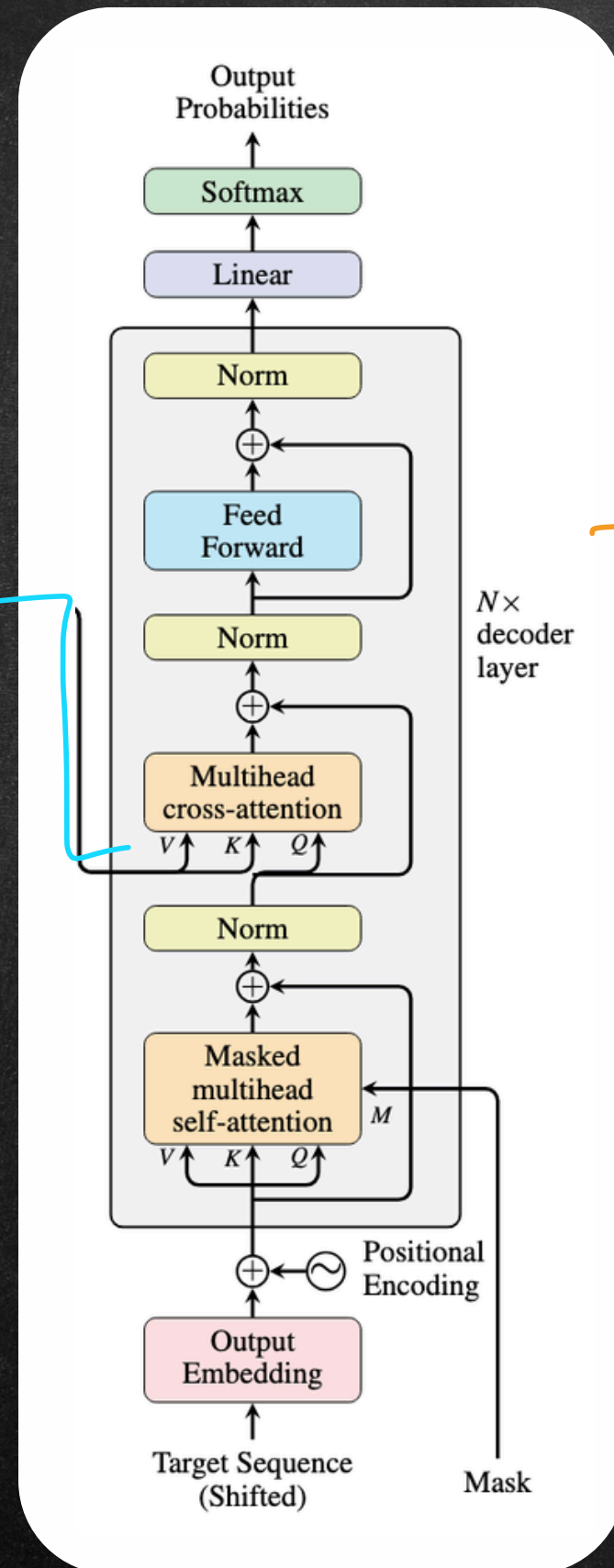
CROSS-ATTENTION

↓
attention btw 2 diff sequences

APNA
COLLEGE

I → ce1
like → ce2
python → ce3

contextual
embedding



"I like Python"

<START>

→ $\frac{7}{10}$

Q: next word?

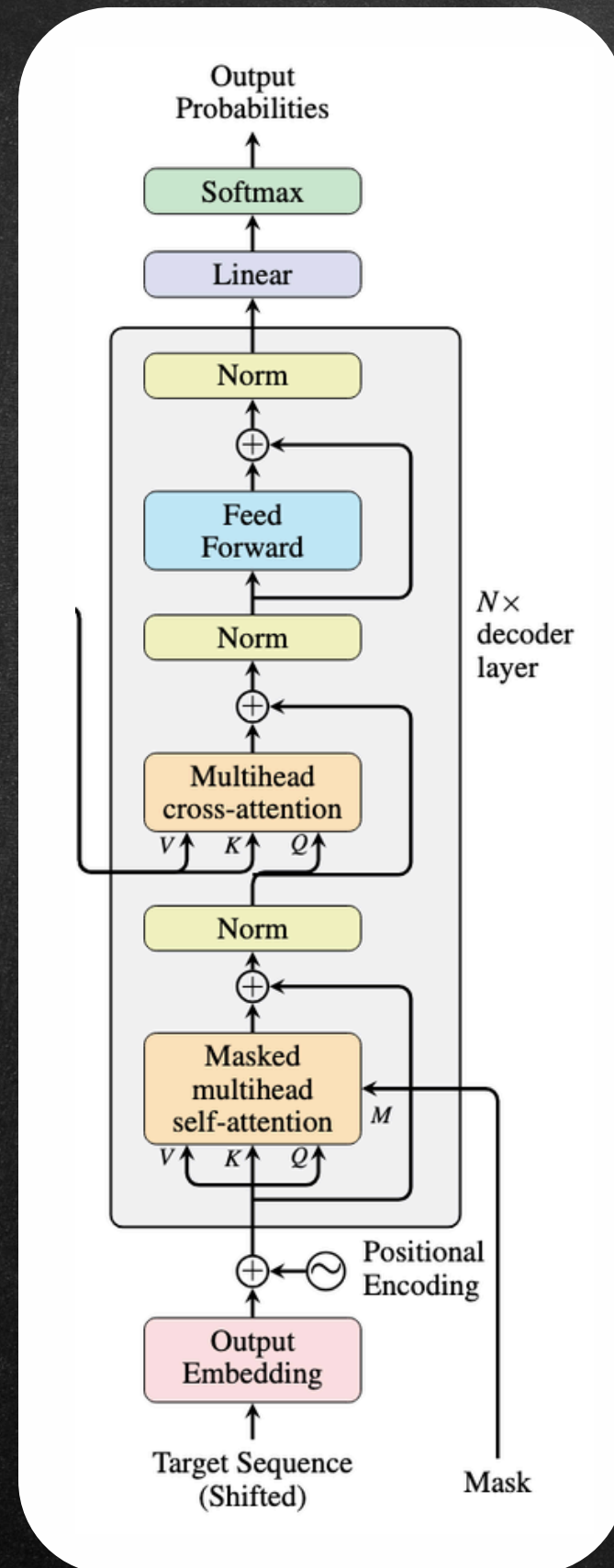
पावना → 80%

|||

Decoder

Masked Attention

⇒ Apples and oranges are fruits.



Training
cheating X

multi-head self-attention
+
mask

generation
word by word



contextual
emb. add

मुझे } प्रश्न
 पढ़ाना } 0/p
 Q: ?
 पसंद
 0/p

बुद्धा — 21.

पसंदा — 72%.

7500 — 10.1-

Pre-trained Transformers

↓
already trained on huge datasets for general tasks

⇓
fine tune

SOTA AI models

→ BERT
GPT-3

T5

Text to Text Transfer
Transformer

self-training

① small dataset → poor generalization ⇒ overfitting ⇒ better generalization

② time ↓ → save

③ money ↓ → billions of params

Pre-trained Transformers

- Hugging Face
- PyTorch Hub
- TensorFlow Hub
- OpenAI APIs etc.